

# 启发式函数

---

祝恩  
计算机学院  
国防科技大学



# 学习内容

---

1. 可采纳性、一致性、准确性  
(启发式函数的特性对搜索算法的影响)
2. 如何设计可采纳的启发式:  
松弛问题、子问题、学习

启发式函数是如何影响搜索算法的？

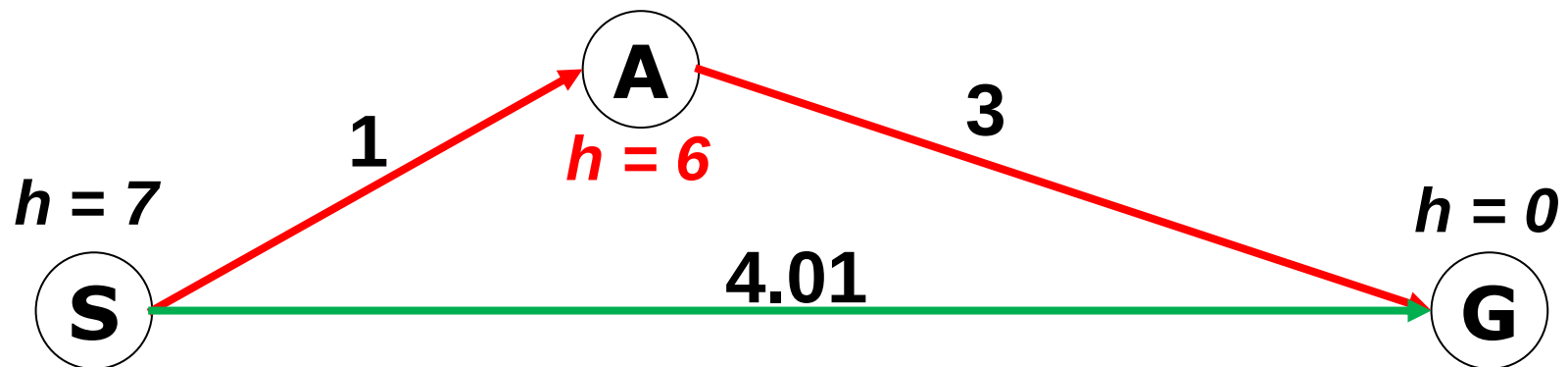
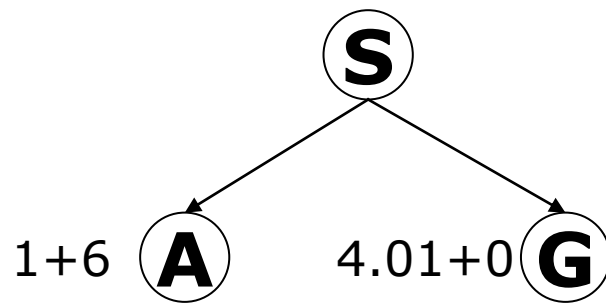
# 1 可采纳性、一致性、准确性

---



## 是否找到最短路径?

□ Try A\* search and find what went wrong?



## 可采纳的启发式函数

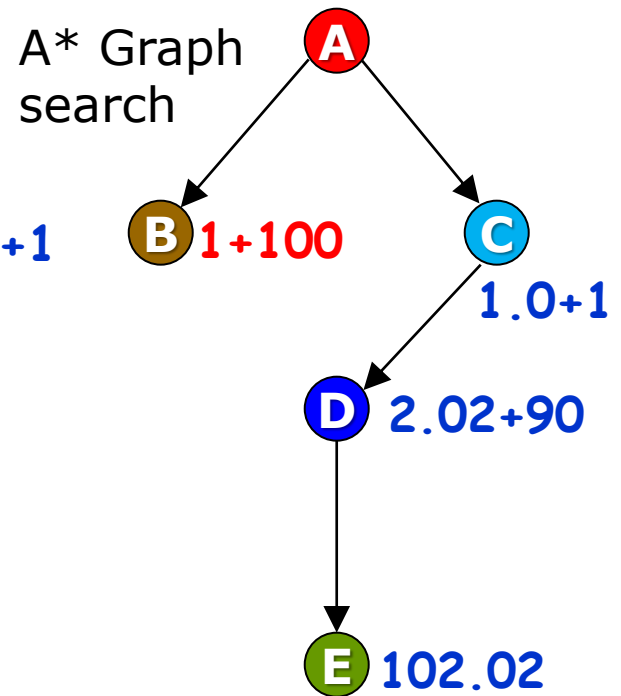
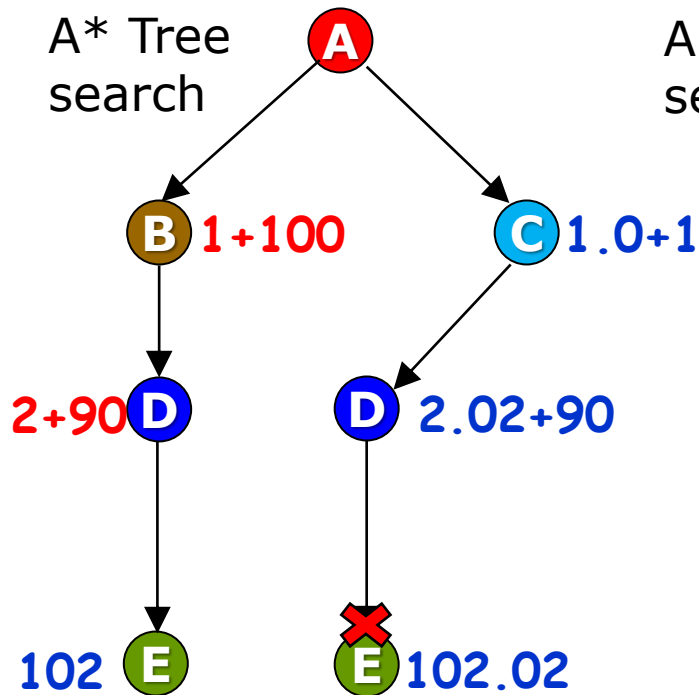
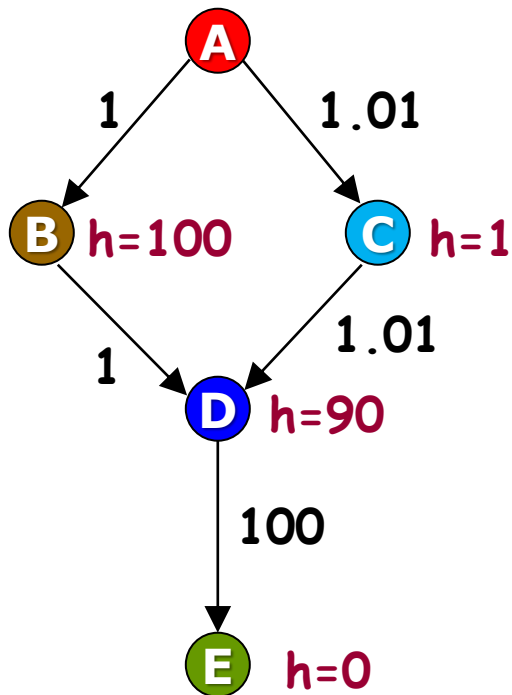
- Let  $h^*(n)$  be the cost of the optimal path from  $n$  to a goal node.
- The heuristic function  $h(n)$  is **admissible** (可采纳的) if:

$$0 \leq h(n) \leq h^*(n)$$

- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic

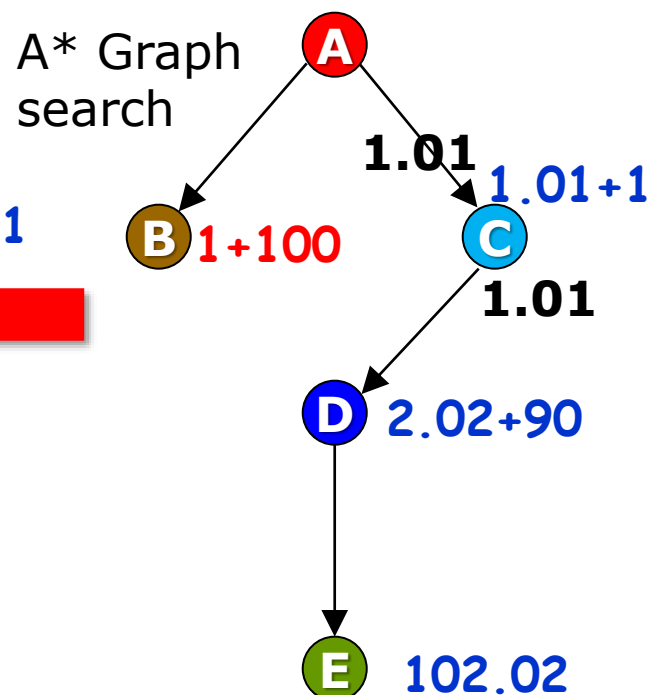
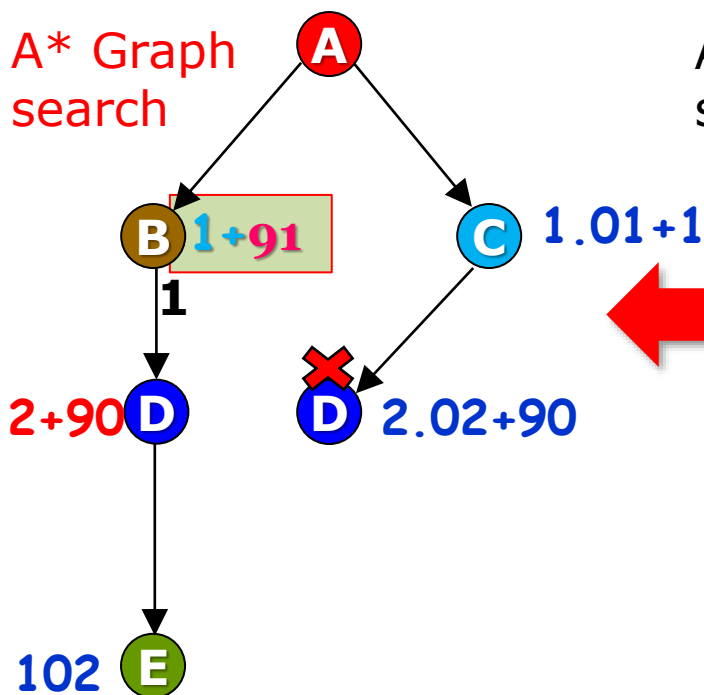
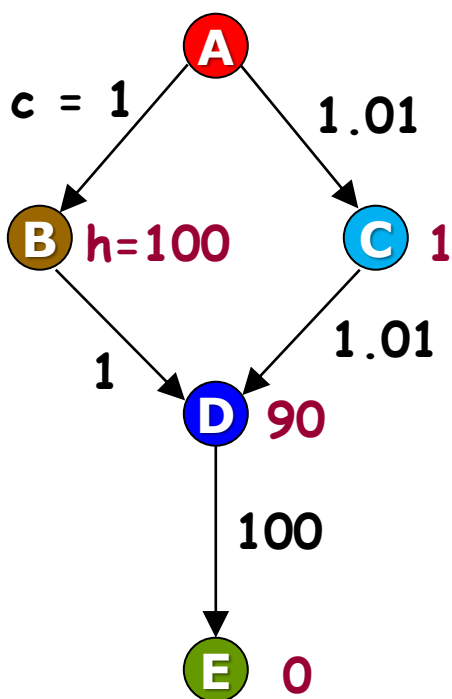
# 是否找到最短路径？

Try A\* tree search and graph search respectively, and find what went wrong:



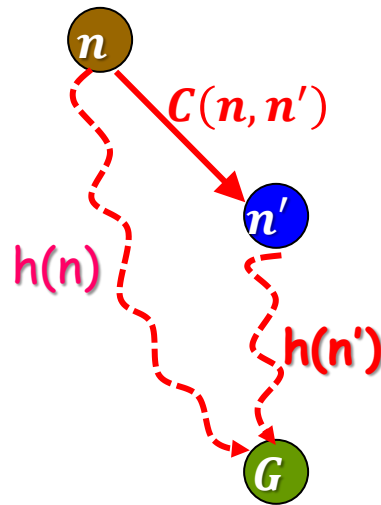
## 施加更强的约束

修改 $h(B)$ , 比低估值更小



$$h(B) \leq C(B, D) + h(D)$$

## 一致性的启发式函数



Consistent heuristic:

$$\forall n, n' \quad h(n) \leq c(n, n') + h(n')$$

Admissible heuristic:

$$\forall n \quad h(n) \leq h^*(n)$$



## 一致的启发式是可采纳的

□ proof: consistent heuristic is admissible.

■  $\forall n, n' \quad h(n) \leq c(n, n') + h(n')$

Entails:  $\forall n \quad h(n) \leq h^*(n)$

□ 假设从 $n$ 到目标 $G$ 的最短路径是

$n, n_1, n_2, \dots, n_k, G$

■  $h^*(n) = c(n, n_1) + c(n_1, n_2) + \dots + c(n_k, G)$

■  $h(n) \leq c(n, n_1) + h(n_1) \leq c(n, n_1) + c(n_1, n_2) + h(n_2) \leq \dots \leq c(n, n_1) + c(n_1, n_2) + \dots + c(n_k, G) + h(G) = h^*(n)$

## A\*搜索是否有最优性?

□ optimal?

|                 | inadmissible | admissible,<br>inconsistent | consistent |
|-----------------|--------------|-----------------------------|------------|
| A* tree search  | <b>no</b>    | <b>yes</b>                  | <b>yes</b> |
| A* graph search | <b>no</b>    | <b>no</b>                   | <b>yes</b> |

C\*是最优解路径的代价

A\* expands \_\_\_\_\_ nodes with  $f(n) < C^*$

A\* expands \_\_\_\_\_ nodes with  $f(n) = C^*$

A\* expands \_\_\_\_\_ nodes with  $f(n) > C^*$

all? some? no?



## A\*搜索的完备性和复杂性

### ☐ Complete?

- Yes, unless there are infinitely many nodes with  $f \leq f(G)$

### ☐ Time?

- Exponential

### ☐ Space?

- Keeps all nodes in memory

## 启发式函数的准确性

two admissible heuristics:

- ❑  $h_1(N)$  = number of misplaced tiles = 6
- ❑  $h_2(N)$  = sum of the (Manhattan) distances of every tile to its goal position =  $3+1+3+0+2+1+0+3 = 13$
- ❑ which one is better?

|   |   |   |
|---|---|---|
| 5 |   | 8 |
| 4 | 2 | 1 |
| 7 | 3 | 6 |

STATE(N)

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

Goal state

## 启发式函数的准确性

- Let  $h_1$  and  $h_2$  be two consistent heuristics such that for all nodes  $N$ :

$$h_1(N) \leq h_2(N)$$

- $h_2$  is said to be **more accurate (or more informed)** than  $h_1$  and is better for search,  $h_2$  **dominates**(占优势)  $h_1$

|   |   |   |
|---|---|---|
| 5 |   | 8 |
| 4 | 2 | 1 |
| 7 | 3 | 6 |

STATE(N)

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

Goal state

- $h_1(N)$  = number of misplaced numbered tiles
- $h_2(N)$  = sum of the (Manhattan) distance of every numbered tile to its goal position
- $h_2$  is more accurate than  $h_1$

## 2 如何设计可采纳的启发式

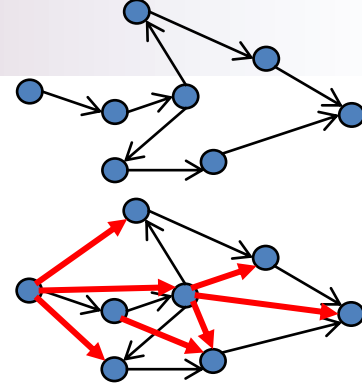
---





## (1) 松弛问题(RELAXED PROBLEM)

## 松弛问题



- 减少了行动限制的问题称为**松弛问题**。  
松弛问题的状态空间图是原有状态空间的超图，原因是减少限制导致图中边的增加。
- 例如，如果八数码问题的行动描述如下：  
棋子可从方格A移动到方格B，如果  
A与B水平或竖直**相邻** 而且 **B是空的**，
- 去掉其中一个或两个条件，生成3个松弛问题：
  1. 棋子可从方格A移动到方格B，如果A和B相邻。(h2)
  2. 棋子可从方格A移动到方格B，如果B是空的。
  3. 棋子可从方格A移动到方格B。(h1)



## 松弛问题的最优解代价 是原问题的可采纳的启发式

- 松弛问题增加了状态空间的边。所以，一个松弛问题的最优解代价一定小于等于原问题的最优解代价，从而是原问题的可采纳的启发式。
- 松弛问题本质上要能够不用搜索就可以求解

## 松弛问题的最优解代价 也是原问题的一致启发式

- 松弛问题获得的启发式是松弛问题的最优路径的解代价，那么它一定遵守三角不等式，因而是一致的

对于任何问题都有  $h^*(n) \leq C(n, a, n') + h^*(n')$  ,

对于松弛问题也成立:  $h_{Relax}^*(n) \leq C_{Relax}(n, a, n') + h_{Relax}^*(n')$

松弛问题的最优路径解代价  $h_{Relax}^*(n)$  是原问题的启发式  $h(n)$

$$h(n) = h_{Relax}^*(n), \quad h(n') = h_{Relax}^*(n')$$

所以:

$$h(n) \leq C_{Relax}(n, a, n') + h(n')$$

松弛问题的动作代价小于等于原问题的动作代价

$$C_{Relax}(n, a, n') \leq C(n, a, n')$$

所以:

$$h(n) \leq C(n, a, n') + h(n')$$

任何问题的最优路径解代价遵守三角不等式。  
松弛问题的最优解的解代价也遵守三角不等式。

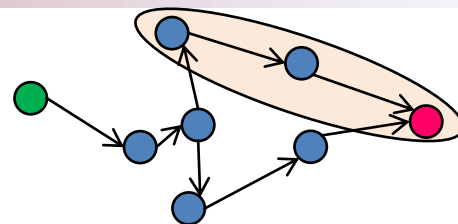
## 多个可采纳的启发式怎么选: max

- 如果可采纳启发式的集合 $h_1, \dots, h_m$ 对问题是有效的, 并且其中没有哪个比其他的更有优势, 我们应该怎样选择?
- 可以定义新的启发式从而得到其中最好的
  - $h(n) = \max\{h_1(n), \dots, h_m(n)\}$



## (2) 子问题

# 子问题



- 子问题的解代价(计算数字的移动以及\*的移动总次数)小于完整问题的解代价，因而是可采纳的。某些情况下比曼哈顿距离更准确。

子模式

|   |   |   |
|---|---|---|
| * | 2 | 4 |
| 3 |   | 1 |
| * | * | * |

目标模式

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | * |
| * | * | * |

1-2-3-4子问题: 只要1,2,3,4到达正确位置

## 构造模式数据库

- **模式数据库**：对每个可能的子问题实例**存储解代价**。八码问题中，每个子问题实例可以是4个棋子和一个空位组成的可能状态。另外4个棋子的移动也计算在解代价里。
- 接着，对搜索中遇到的每个完整状态，在数据库里**查找**出相应的子问题，得到其可采纳的启发式  $h_{DB^o}$

|   |   |   |
|---|---|---|
| 3 | 1 | 2 |
| 4 | * | * |
| * |   | * |

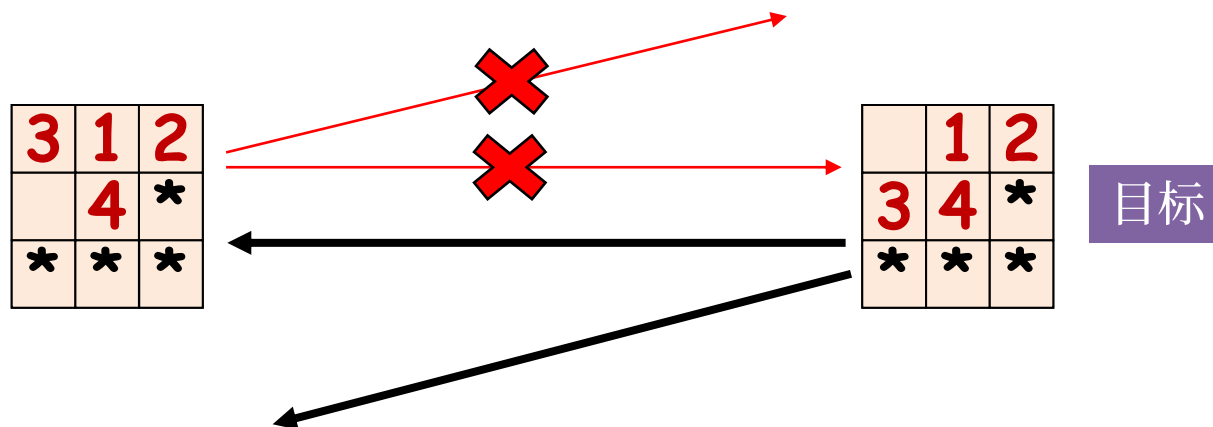
**h=?**

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | * |
| * | * | * |

目标

# 构造模式数据库的高效方法

- 数据库本身的构造是通过从目标状态**向后搜索**并记录下每个遇到的**新**模式的代价完成的；搜索的**开销分摊**到许多子问题实例上。



## 多个模式数据库

- 可以构造多个模式数据库，例如1-2-3-4的模式数据库，5-6-7-8的模式数据库，2-4-6-8等的模式数据库。

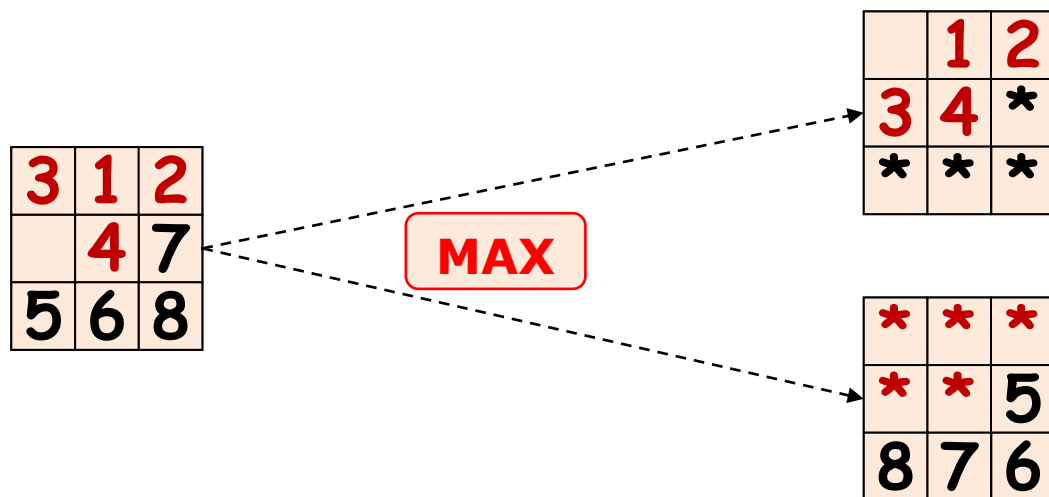


考虑利用多个模式数据库构造比单个模式数据库更精确的可采纳的启发式。



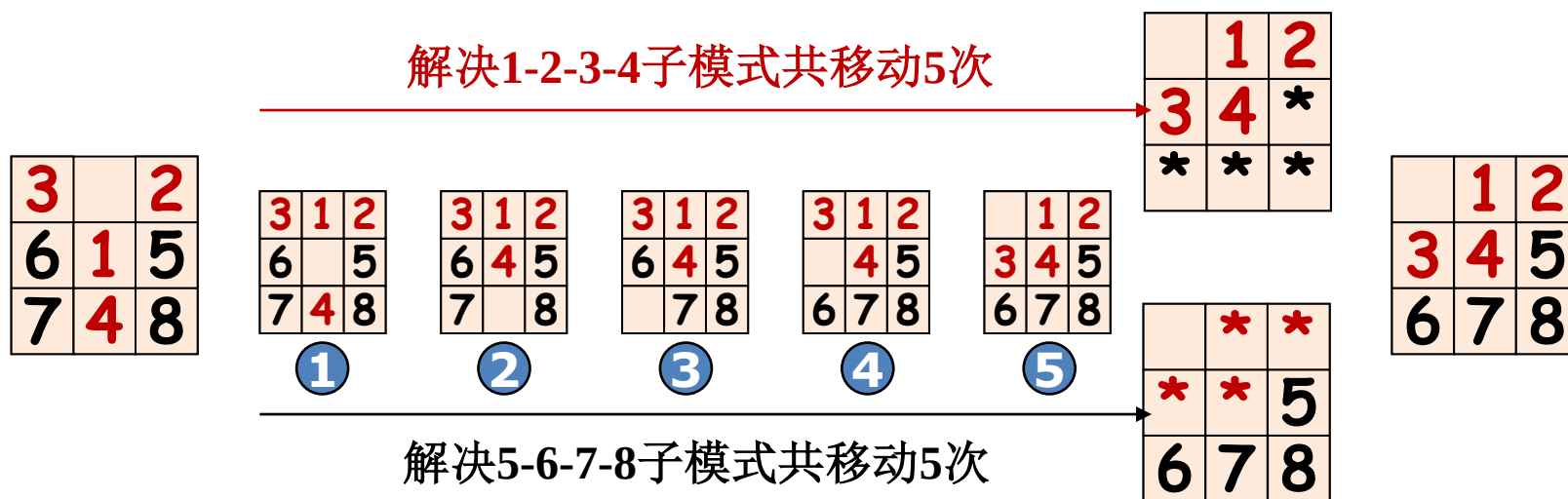
### 多个模式数据库如何组合: max

- 每个数据库都能产生一个可采纳的启发式，这些启发式可以像前面所讲的那样**取最大值的**方式组合使用。
- 这种组合的启发式比曼哈顿距离要精确；求解随机的15数码问题时所生成的结点数要少1000倍。



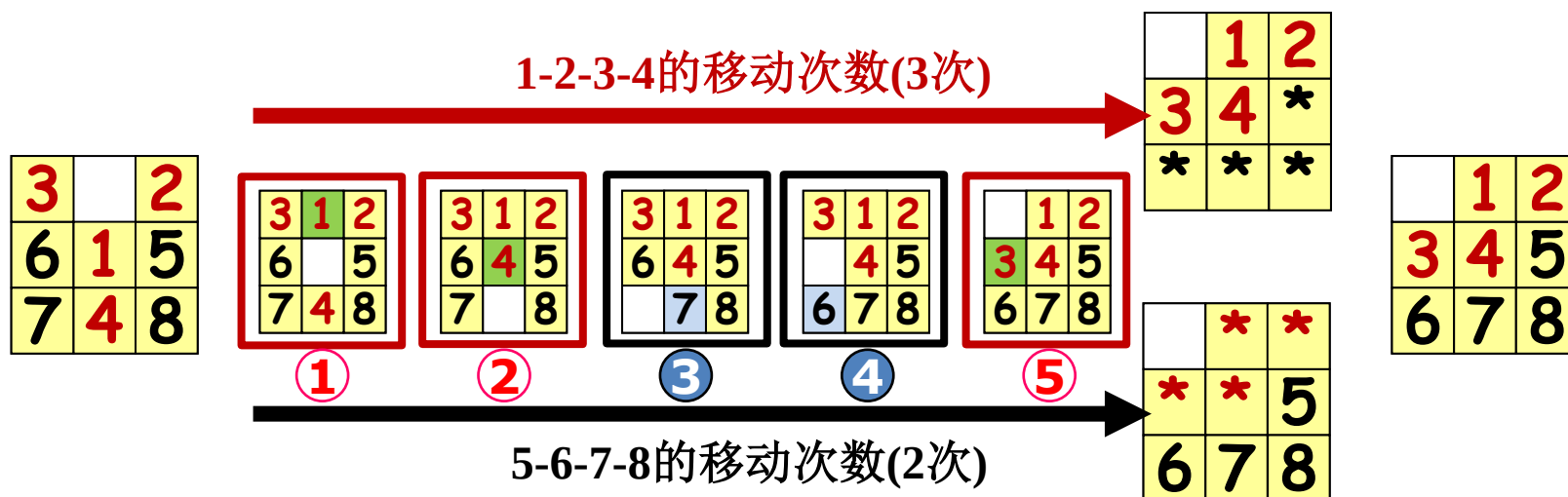
## 考虑相加 – 不是可采纳的

- 相加得到的启发式不是可采纳的，因为对于一给定状态，1-2-3-4子问题的解和5-6-7-8子问题的解可能有一些重复的移动



## 不相交的模式数据库

- 1-2-3-4子问题只记录涉及1-2-3-4的移动次数。5-6-7-8子问题只记录涉及5-6-7-8的移动次数。这样很容易得出，两个子问题的代价之和仍然是求解整个问题的代价的下界。这就是不相交的模式数据库的思想。
- 用这样的数据库，可以在几毫秒内解决一个随机的15数码问题——与使用曼哈顿距离启发式相比生成的结点数减少了10,000倍。对于24数码问题减少的结点数以百万倍计。

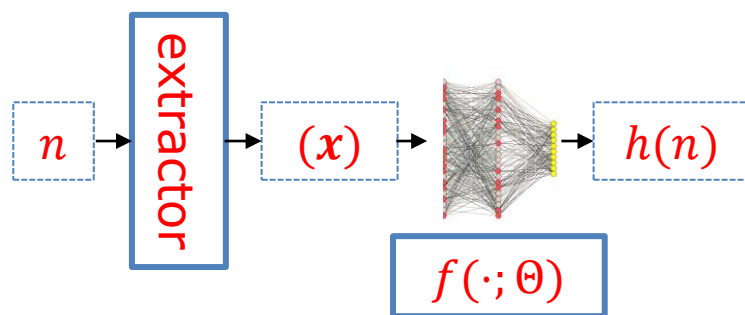




## (3) 学习

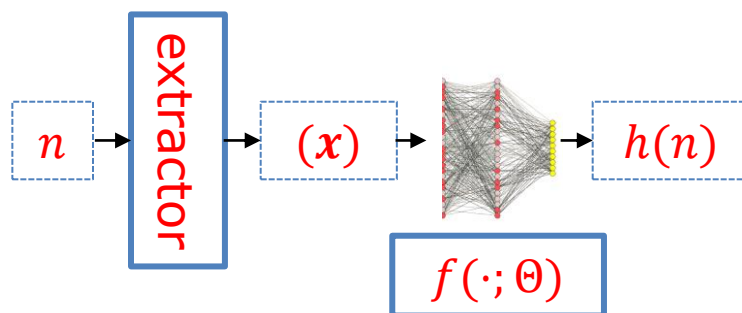
## 学习启发式

- 另一个方案则是从经验（求解大量八码问题）里学习。每个八数码问题的最优解都可提供学习  $h(n)$  的实例。每个实例都包括解路径上的一个状态和从这个状态到达解的代价。一个学习算法从实例构造  $h(n)$ ，预测搜索过程中所出现的其他状态的解代价。



## 学习启发式

- 如果在状态描述外还能刻画给定状态的**特征**，**归纳学习方法**则是最可行的。例如，特征“**不在位的棋子数**”  $x_1(n)$ ，“**现在相邻但在目标状态中不相邻的棋子对数**”  $x_2(n)$ ，用线性组合构造  $h(n)$ ：
- $h(n) = c_1 x_1(n) + c_2 x_2(n)$ 。
  - $c_1$ 和 $c_2$ 是需要学习的常数
  - 这个启发式满足目标状态  $h(n) = 0$ ，**但不保证是可采纳性或一致性。**



**Thanks**  
LU9UK2